Type Safety.

a) **Audit Report**

The type-safety audit was performed using static analysis over the TypeScript codebase (api/src, web/src, shared/src) with compiler-AST parsing to identify explicit any, type assertions (as), non-null assertions (!), untyped function parameters, and missing function return type annotations, with directive checks for @ts-ignore/@ts-expect-error. Results were aggregated by package and violation type, then ranked by file-level density. Strict mode is enabled across project configurations. For remediation prioritization under strict mode, test/spec files were excluded from Top 5 density ranking to focus on production-impacting paths.

The Top 5 violation-dense production files indicate concentration in high-impact UI orchestration and core API route modules. web/src/components/IssuesList.tsx and web/src/pages/App.tsx are dominated by missing return annotations and untyped parameters, reducing type guarantees in heavily reused interaction surfaces and application shell logic. api/src/routes/weeks.ts and api/src/routes/issues.ts combine missing return annotations with substantial non-null assertions, increasing the risk of nullability-related runtime faults in central backend workflows. web/src/pages/ReviewsPage.tsx shows similar density in a complex workflow page where weaker typing can mask state-transition defects and make regressions harder to detect during refactors.

b) **Remediation**

The remediation effort across seven commits targeted the highest-density production files identified in the baseline audit, reducing total violations from 5,540 to 3,900 (−29.7%). The largest gains came from eliminating type assertions (as casts reduced 39.5%) and non-null assertions (! operators reduced 49.9%), particularly in the API route modules weeks.ts and issues.ts where nullable database row access was tightened with explicit RequestContext interfaces and typed extraction functions. On the frontend, missing return type annotations were the dominant fix category (−517 instances), applied systematically to React hooks, useEffect callbacks, event handlers, and component signatures in IssuesList.tsx, App.tsx, Editor.tsx, and supporting pages. Query hooks (useIssuesQuery, useProjectsQuery, useWeeksQuery, useDocumentsQuery) were refactored with typed mutation context and response interfaces to replace implicit any propagation through React Query's generic boundaries. The commit sequence was structured to address backend routes and frontend orchestration surfaces in alternating passes, allowing the audit tooling to confirm incremental violation counts between each change and verify that the 25% production-violation reduction target was met without introducing regressions.

Table 1. Type safety report: baseline (5,540) vs current (3,900) type safety errors.

Metric	Baseline	Current
Total any types	260	250
Total type assertions (as)	691	418

Total non-null assertions (!)	321	170
Total @ts-ignore / @ts-expect-error	1	1
Strict mode enabled?	Yes	Yes
Strict mode error count (if disabled)	N/A	N/A
Top 5 violation-dense files	components/IssueList.tsx (189), pages/App.tsx (180), routes/week.ts (159), issues.ts (159), pages/ReviewsPage.tsx (150)	pages/OrgchartPage.tsx (79), collaboration/index.ts (72), routes/issues.tsx (70), components/IssueList.tsx (68), db/seed.ts (79)

Bundle Size.

a) **Audit**

The production bundle was built using Vite with rollup-plugin-visualizer (enabled via ANALYZE=true) to generate an interactive treemap and structured size data. All sizes were extracted programmatically from the visualizer's rendered-length metric (post-treeshake, pre-minify) via an automated script at scripts/bundle-size-report.mjs. The total bundle is 4,771 KB rendered (1,205 KB gzip / 1,028 KB brotli) across 261 chunks. The main entry chunk (index-*.js) dominates at 4,532 KB rendered (95% of the total). The remaining 260 chunks, mostly USWDS icon SVGs lazy-loaded via import.meta.glob and a handful of document tab components split via React.lazy — account for only 239 KB combined. The top three dependencies by size are emoji-picker-react (481 KB, 10.1%), highlight.js (378 KB, 7.9%), and Yjs (265 KB, 5.6%). One dependency is unused (@tanstack/query-sync-storage-persister), and one is misplaced (@tanstack/react-query-devtools is a production dep but only provides dev tooling). The full dependency breakdown, treemap visualization, and automated script are committed under reports/bundle-size/.

Existing code splitting is limited to two areas that are already working well: USWDS icons are lazy-loaded at the component level via import.meta.glob, and several document tab components (e.g., ProgramWeeksTab, WeekReviewTab) are split via React.lazy in document-tabs.tsx. However, all 20 page-level routes in main.tsx are statically imported, which forces every page, the full TipTap/ProseMirror editor stack, and all their transitive dependencies into the single main chunk. The highest-impact improvement would be converting page imports in main.tsx to React.lazy() calls — particularly for heavy, infrequently-accessed pages like AdminDashboardPage, OrgChartPage, WorkspaceSettingsPage, PersonEditorPage, and FeedbackEditorPage, which most users never visit in a given session. Beyond route-level splitting, two large dependencies are strong candidates for lazy loading: emoji-picker-react (481 KB) is only used behind a click interaction in EmojiPicker.tsx and could be dynamically imported on first open, and highlight.js (378 KB) is pulled in by the TipTap code-block extension and could

be deferred until a user actually encounters or creates a code block. Together, these two libraries account for 18% of the bundle and are rarely needed on initial page load.

b) Remediation

The optimization commit converted all 16 statically-imported page routes in main.tsx to React.lazy() calls wrapped in a shared Suspense boundary via withRouteSuspense(), moving pages like AdminDashboardPage, OrgChartPage, WorkspaceSettingsPage, PersonEditorPage, ReviewsPage, and UnifiedDocumentPage out of the initial chunk and into on-demand route-level splits. The emoji-picker-react library (481 KB) was changed from a static import to a dynamic import() triggered on first popover open in EmojiPicker.tsx, with a loading placeholder shown while the module resolves. On the Vite/Rollup side, a manualChunks configuration was added to vite.config.ts to isolate three vendor groups into dedicated chunks: editor-core (TipTap, ProseMirror, Yjs, y-websocket, y-indexeddb, lib0 — 1,798 KB), syntax (highlight.js, lowlight — 390 KB), and emoji (emoji-picker-react — 491 KB). Additionally, @tanstack/react-query-devtools was changed from a static production import to a import.meta.env.DEV-gated React.lazy() load, eliminating it entirely from production builds. Together these changes reduced the initial entry chunk from 4,532 KB (95% of total) to 857 KB (18% of total), increased chunk count from 261 to 288, and brought the initial load under the target budget of 3,626 KB rendered — while the total bundle size remained stable at 4,776 KB since no code was removed, only redistributed across lazy-loaded boundaries.

Table 2. Bundling report.

Metric	Baseline	Current
Total production bundle size	4,771 KB	4,475 KB
Bundle size on initial load	4,771 KB	856.9 KB
Largest chunk	index-oq$4spmW.js (4,532 KB)	assets/editor-core-BFf19B0r.js (1,798 KB)
Number of chunks	261 chunks	288
Top 3 largest dependencies	Emoji-picker-react (481 KB)	
	highlight.js (378 KB)	
	Yjs (265 KB)	
Unused dependencies identified	tanstack/query-sync-storage-persister	0

Response Time.

a) Audit

A repeatable API response-time audit was executed against a dedicated, deterministic performance dataset rather than an empty database. The benchmark seed created and verified realistic volume in a separate workspace (Perf Benchmark Workspace): 665 documents, 160 issues, 24 users, and 24 sprints (all above the required thresholds). The five benchmark endpoints were selected from confirmed frontend common-flow usage (app shell lists plus My Week): /api/documents?type=wiki, /api/issues, /api/projects, /api/programs, and /api/dashboard/my-week. Load testing used autocannon with a 15-second warmup and 30-second measured runs at 10, 25, and 50 concurrent connections and capturing P50/P95/P99, throughput. After enabling benchmark-mode rate-limit settings (API_BENCHMARK=1), all measured runs returned non_2xx = 0, so reported latency reflects successful responses.

The slowest endpoints were consistently /api/documents?type=wiki and /api/issues, with highest tail latency at concurrency 25 (P95 57ms and 54ms, respectively). This ranking aligns with query complexity visible in route implementations. The documents list route applies workspace/visibility filtering, optional type filtering, and ordered output (position, created_at) over larger result sets, which increases sort/filter cost as wiki volume grows. The issues list route performs broader relational work: multiple joins (assignee/person), optional association filters, and per-row association hydration (getBelongsToAssociationsBatch) plus priority-based ordering; this creates higher CPU/DB work per response than simpler list routes. By contrast, /api/programs and /api/projects remained faster in this dataset despite aggregate/count logic, and /api/dashboard/my-week stayed mid-tier, indicating that under current seeded shape and indexing behavior, the dominant tail-latency contributors are the high-volume document and issue list paths.

b) Remediation

The optimization work spanned three commits targeting the request path, payload shape, and query consolidation. In documents.ts, the SELECT column list for the wiki list endpoint was narrowed by dropping workspace_id and conditionally omitting ticket_number, properties, updated_at, and created_by depending on the requested view; the original query returned all columns regardless of context, which inflated serialization cost on every response. A new view=tree query parameter was introduced so the sidebar tree fetch (which only needs id, title, parent_id, position, and visibility) sends a lighter payload than the full documents table view, reducing wire bytes and JSON.stringify time for the dominant wiki list call. On the frontend, useDocumentsQuery was split into two cache variants (tree and full) with separate React Query keys, so the sidebar and table each hit the appropriately scoped endpoint without over-fetching; optimistic mutation helpers (setWikiListData) update both caches simultaneously to keep them consistent. In issues.ts, a server-side project_id filter was added via an EXISTS subquery on document_associations, replacing the previous approach where the client fetched all issues and filtered by project association in JavaScript — this was suboptimal because it transferred the full issue set over the wire when only a project-scoped subset was needed. In the request-performance middleware, the res.json override was rewritten to call JSON.stringify once via a custom stringifyJsonResponse and pass the result to res.send, eliminating Express's default double-serialization path where res.json internally calls JSON.stringify and then res.send

processes the result again; the Server-Timing header injection was also moved from writeHead-only to a shared applyServerTiming guard on both writeHead and end, fixing cases where timing data was lost on responses that skipped writeHead. For the /api/dashboard/my-week endpoint, two sequential queries (person lookup and workspace config) were consolidated into a single query using a LEFT JOIN LATERAL to fetch the person document inline with the workspace row, and three separate weekly document queries (plan, retro, previous retro) were collapsed into one query using document_type IN ('weekly_plan', 'weekly_retro') with (properties->>'week_number')::int = ANY($3) over an array of target week numbers; the original code issued five sequential round-trips where two suffice, and the compounding latency was visible in the db_main phase breakdown. The tradeoff across all changes is increased query and caching complexity — conditional column lists, dual cache keys, and lateral joins are harder to reason about than their simpler predecessors — but the benchmark results justify it: at concurrency 25, P95 latency dropped from 57ms to 38ms on /api/documents?type=wiki (−33.3%), from 54ms to 40ms on /api/issues (−25.9%), and from 39ms to 23ms on /api/dashboard/my-week (−41%), with all five endpoints meeting the 20% improvement gate.

Table 3. API response report at concurrency (25).

Endpoint	P50 (ms) baseline	P50 (ms) current	P95 (ms) baseline	P95 (ms) current	P99 (ms) baseline	P99 (ms) current
documents?type=wiki	47	30	57	38	60	40
issues	44	32	54	40	57	43
projects	21	9	32	16	37	11
programs	15	6	24	9	30	11
dashboard/my-week	24	13	39	23	42	25

Database Query Efficiency.

a) Audit

Database query efficiency was evaluated by tracing the live application routes used by the frontend and capturing the SQL actually executed by the backend during five common user flows: loading the main page, viewing a document, listing issues, loading the sprint board, and searching content. Query counts were measured per flow, and EXPLAIN ANALYZE was run on the observed read queries to review the slowest statements and their execution plans. This approach was used to assess the unified document model for N+1 query behavior, sequential scans, missing or ineffective index coverage, and unnecessary data retrieval.

The results showed that the most query-intensive flow was viewing a document, which executed 32 database queries and was the only flow with confirmed N+1 behavior. That finding was caused by correlated subqueries in the supporting programs and projects requests loaded with

the document page, where related counts and status values were calculated once per outer row instead of through fully set-based aggregation. Other flows did not show confirmed N+1 behavior in the final run, but the issues flow and sprint board flow still showed inefficient broad retrieval, including repeated loading of large properties payloads and list-level fetching of fields not clearly required for the immediate view.

Execution-plan review also identified problematic filter patterns. Repeated sequential scans were observed on document_associations predicates involving relationship_type and document_id, indicating that the existing index strategy is not serving those traced access patterns efficiently. The clearest likely missing-useful-index case was the content search predicate on documents.title ILIKE '%...%', combined with workspace_id, document_type, and deleted_at, because substring matching of that form is typically not well supported by ordinary indexes. Overall, the audit found that the main database efficiency risks are concentrated in document-page enrichment queries, inefficient association-table filtering, and unnecessary data fetching from the unified documents table.

b) remediation

Three route-level query rewrites addressed the confirmed N+1 behavior and correlated-subplan inefficiency identified in the baseline audit. In programs.ts, the original query computed issue_count and sprint_count using two correlated scalar subqueries,(SELECT COUNT(*) FROM documents i JOIN document_associations da ...), embedded in the outer SELECT, which the planner executed once per program row (8 executions each at baseline). The rewrite replaced these with two CTEs (program_issue_counts, program_sprint_counts) that each aggregate document_associations joined to documents in a single pass grouped by related_id, then LEFT JOIN the pre-aggregated results back to the outer query; this is better because the planner can hash-join the materialized aggregates rather than re-scanning the association table per row, and the EXPLAIN ANALYZE confirmed the correlated SubPlan nodes were eliminated. The same pattern was applied in projects.ts, which had three correlated subplans per outer row (sprint count, issue count, and inferred allocation status computed by scanning all sprint documents per project). The rewrite lifted all three into CTEs, project_sprint_counts, project_issue_counts, and project_allocation_status, with the allocation status CTE pre-computing the max sprint-timing rank per project in a single scan of sprint documents filtered to the workspace, then joining the result. This eliminated the 24×-per-row SubPlan executions that the baseline audit flagged on the document-view flow. In issues.ts, the original query performed a LEFT JOIN to users and documents (person lookup) directly on every issue row to resolve assignee names and archived status; the rewrite restructured this as a CTE pipeline where filtered_issues first narrows to the matching issue set, assignee_lookup extracts the distinct assignee_id values, and assignee_details joins users and person documents only for those distinct IDs, this avoids redundant person-document lookups when multiple issues share the same assignee. For the /api/dashboard/my-week endpoint, a separate commit consolidated the person and workspace lookup from two parallel queries into a single LEFT JOIN LATERAL query, and collapsed three separate weekly-document queries (plan, retro, previous retro) into one using document_type IN (...) with (properties->>'week_number')::int = ANY($3), reducing the flow's query count from 9 to 6. The tradeoff is that CTE-based rewrites

produce more complex SQL and can occasionally prevent the planner from pushing predicates into the CTE (PostgreSQL materializes CTEs by default before version 12, though this project runs 14+ where the planner can inline them). The measured results show the "List issues" flow improved the most dramatically,total DB time dropped from 18.68ms to 3.52ms (−81.2%) and the slowest query fell from 15.93ms to 1.75ms (−89%), while the "View a document" flow dropped from 26.47ms to 19.47ms (−26.4%) with all five baseline N+1 findings resolved, and the "Load main page" flow dropped from 39.21ms to 24.06ms (−38.6%) with three fewer queries.

Table 4. Database query efficiency report.

User Flow	Total Q baseline	Total Q current	Slowest Q (ms) baseline	Slowest Q (ms) current	N+1 baseline	N+1 current
Load main page	9	6	9.18	10.58	No	No
View a document	24	24	4.32	3.78	Yes	Yes
List issues	4	4	15.93	1.75	No	No
Load sprint board	17	17	1.69	1.63	No	No
Search content	5	5	.57	.63	No	No

Test Coverage and Quality.

a) Audit

The audit ran the full measured test workload across unit and end-to-end suites and produced a usable baseline. In total, 1,474 tests were counted, with 1,439 passing, 17 failing, and 5 identified as flaky across repeated runs. The full run took about 605 seconds. Code coverage is configured and reporting at a relatively modest level: the API is a little over 40% line coverage and 33% branch coverage, while the web app is under 30% line coverage and under 20% branch coverage.

On the specific question of “critical flows with zero coverage,” the four flows explicitly called out in the audit brief do not appear to be completely untested. Document creation and editing, real-time collaboration, authentication, and sprint-management workflows all have direct evidence in the current suite, with both backend and browser-level coverage present in most cases.

The deeper review found that the real zero-coverage risk sits in operational and secondary user journeys outside those four headline flows. The clearest examples are the first-run setup experience, the CAIA/PIV authentication path, the admin flow for managing CAIA credentials,

workspace API token management, the converted-documents history screen, the person profile editor, and the org-chart interaction flow. These are all real product surfaces with meaningful business or operational impact, but no targeted behavioral tests were found for them. That is the most useful audit conclusion for a stakeholder: the core collaboration product is not untested, but there are several important onboarding, admin, auth-variant, and people-management flows that currently have little to no meaningful test protection.

b) Remediation

The primary coverage addition targeted routes/weekly-plans.ts, which the audit identified at 5.09% line coverage with zero end-to-end tests despite powering weekly plans, retros, and the project allocation grid, APIs used by every team member each sprint. Three new end-to-end test suites were added covering the weekly plans flow (CRUD operations, create-idempotency, list filtering, project association, input validation, 404 handling, and authentication gates), the weekly retros flow (bare template creation, plan-reference template creation, retro-idempotency, and content cross-referencing), and the allocation grid flow (grid structure verification, status transitions, and authorization checks), twenty-six tests in total. The original gap existed because this route was added after the initial test suite was written and no coverage followed it in. The new tests exercise the full request lifecycle through the running app rather than mocking the handler, so they catch regressions in middleware, database queries, and response shape simultaneously. The tradeoff is that these are integration-speed tests against a seeded database, so they are slower than unit-level route tests would be, but they match the actual contract the frontend depends on and are more likely to catch the category of drift that caused the existing failures.

The remaining fixes addressed three unit test files that were already failing before the audit began, all due to test-implementation drift rather than production bugs. The document-tabs.test.ts suite was asserting an older tab model, expecting a sprints tab on programs and no tabs on sprint documents, while the implementation had moved to week-based tabs for programs and status-dependent tab sets for sprints; the fix updated assertions to match the current contract and added coverage for planning-versus-active sprint tab selection. The DetailsExtension.test.ts suite was constructing a TipTap editor with only the DetailsExtension node, but the extension now requires detailsSummary and detailsContent child node types to build a valid schema; the fix registered those companion extensions in the test editor and asserted on the correct inserted node shape. The useSessionTimeout.test.ts suite mocked fetch as though resetTimer() made a single request, but apiPost now fetches a CSRF token before sending the POST, so the incomplete mock caused the CSRF path to fail and the hook to fire onTimeout; the fix mocked the real two-request sequence, CSRF token fetch then extend-session POST, with JSON headers so the API helper treated both responses as valid. No production code was changed to resolve any of these failures; the corrections were entirely in test expectations and test scaffolding that had fallen out of sync with the codebase. The tradeoff in all three cases is that the tests are now tightly coupled to current implementation details (tab names, node schema, request sequence), which means future refactors will need to update tests in tandem, but that is preferable to a green suite that silently tests nothing.

Table 5. Test coverage and quality report.

Metric	Baseline	Current
E2E (Pass / Fail / Flaky)	864/7/3	874/7/1
Web (pass / total)	143/156	159/159
API (pass / total)	459/459	459/459
Suite runtime	30 mins	30 mins
Critical flows with zero coverage	Initial setup, CAIA/PIV Auth, admin credential management, org chart, person profile editor, converted documents history	Added e2e tests for Weekly plans, Weekly retros, Allocation grid. (26 total tests)
Code coverage %	web: 28.53% lines / 19.38% branches / api: 40.53% lines / 33.45% branches	Web: 29.89% lines, 23.02% branches / API: 41.6% lines, 35.51% branches

Runtime Error and Edge Case Handling.

b) Audit

The audit focused on Category 6 runtime resilience by exercising normal app usage, document navigation, collaboration, malformed input, slow-network behavior, and failure recovery in a local seeded environment. Browser DevTools behavior was reproduced through automated browser sessions against the running dev app, while the API terminal was monitored in parallel to capture backend errors. The test coverage included route transitions across docs, issues, programs, projects, weekly plans, weekly retros, standups, reviews, search flows, legacy redirect paths, offline and reconnect behavior, and multi-tab collaboration against the same document.

The method was to compare visible UI behavior against both browser-console output and server logs for each workflow. That made it possible to separate true crashes from handled backend

errors, console noise, silent failures, and blocked interactions. The results showed repeatable handled server errors around weekly AI analysis, collaboration-related browser warnings, partial offline recovery, a concurrent title-sync bug, and an interaction bug where the Action Items modal backdrop can block issue-list clicks. No true process-level unhandled promise rejection or proven infinite spinner was observed in the audited scenarios, but several async failure paths degraded quietly instead of surfacing explicit user-facing errors.

b) Remediation

The title deconflict fix changed how Editor.tsx and UnifiedEditor.tsx handle concurrent title mutations. The original code applied every incoming document:updated event directly to the title field, which meant a remote user's edit could overwrite an in-progress local edit before it was committed — the input felt unstable and the saved value was nondeterministic. The fix introduced a hasLocalChangesRef guard that tracks whether the local user has uncommitted keystrokes; incoming remote title updates are only applied when that flag is false, and local edits fire onTitleChange immediately on input rather than waiting for blur so that other connected users receive the update promptly. The supporting infrastructure added PostgreSQL LISTEN/NOTIFY and split the broadcast layer into local-only and cross-process paths so title sync works correctly even across multiple API server instances. The tradeoff is a brief window where two users editing the same title simultaneously will each see their own version until one commits, at which point the other receives the update — last-write-wins still applies, but the user is no longer surprised mid-keystroke.

The forbidden-conversion fix moved permission enforcement from a backend-only 403 rejection into a frontend-aware gate. The original code rendered conversion actions (issue → project, project → issue) unconditionally and only discovered the restriction when the API returned a 403, which was then misidentified as a possible CSRF failure and retried — producing duplicate error noise and a confusing user experience. The fix introduced a documentConversion.ts utility that checks whether the current user is the document creator before rendering the action, disables the conversion button with a clear "Only the document creator can convert this document" explanation when they are not, and added isCsrfErrorResponse() to the API client so that generic 403s are no longer conflated with CSRF token failures. The tradeoff is that conversion eligibility is now coupled to the creator identity available on the client at render time, so if the permission model later broadens (e.g., workspace admins can also convert), the frontend gate must be updated alongside the backend.

The retro AI availability fix replaced a silent failure path with an explicit three-state model. The original code treated the AI analysis endpoint as either working or errored, but when the backend returned { error: 'ai_unavailable' }, the UI had no distinct handling — it implied feedback would appear as the user wrote, even though no analysis could run. The fix introduced an AiAvailabilityState type (checking | ready | unavailable) and updated RetroQualityBanner and QualityAssistant to show a non-blocking message ("AI feedback is temporarily unavailable. You can still write and submit your retro normally.") when the state resolves to unavailable, so the retro workflow is never blocked and the user understands why suggestions are absent. The tradeoff is that the availability check adds one request on component mount and the UI must

handle three states instead of two, but the cost is negligible compared to the confusion of a silently broken assistant panel.

Table 5. Runtime error and edge case handling report.

Metric	Baseline	Current
Console errors during normal usage	401 (login error), 403 (changing document types, upgrade to project)	
Unhandled promise rejections	1 (inconsistent in retro view)	
Network disconnect recovery	Partial	
Missing error boundaries	No hard React render crash escaped during this pass, but async failure handling is weak around weekly AI analysis and auxiliary fetch surfaces. Likely insufficient async/global error handling at main.tsx, weekly AI banner surfaces, and backlink polling.	Fixed: real time title deconfliction between multiple users; addressed forbidden document conversion.
Silent failures identified	Weekly retro AI analysis fails silently: open a weekly retro, server logs Retro analysis error. 3. Concurrent title edits diverge	Fixed: Silent failure of retro AI assistant, now returns AI Unavailable.

Accessibility Compliance.

a) Audit

An accessibility baseline audit was completed across 13 major application pages using automated axe-core scans and Lighthouse accessibility audits. The reporting pipeline was corrected so Lighthouse runs against the current application state, page readiness is checked before scoring, raw Lighthouse JSON is captured per page for traceability, and axe violation totals are counted by affected elements rather than by rule instance. The report was then regenerated and recorded as the baseline.

The baseline results show Lighthouse accessibility scores ranging from 96 to 100 across the audited pages, with an average of 99. Automated scanning found 26 critical or serious affected elements in total: 25 color contrast failures and 1 missing accessible name issue. The missing ARIA/name/role finding was limited to a single Settings page control at tr:nth-child(1) > td:nth-child(3) > select. Manual testing found keyboard navigation completeness to be partial on

the Allocation, Directory, and Org Chart pages, and VoiceOver support was also assessed as partial. The summary table in the report includes the detailed baseline values.

b) Remediation

The single critical violation was a <select> element on the Settings page Members tab that rendered without an accessible name, making the role dropdown invisible to screen readers. The fix added an aria-label attribute (Role for ${member.name || member.email}) to WorkspaceSettings.tsx, giving assistive technology a unique, descriptive label per row rather than a generic one. The 25 serious violations were all WCAG AA color contrast failures concentrated in three patterns. The most widespread was the accent blue (#005ea2, 2.74:1 ratio against dark backgrounds) used for current-week labels, active sidebar buttons, sprint name badges, and status indicators across 10 components — AccountabilityGrid, DashboardSidebar, StatusOverviewHeatmap, DashboardVariantC, ProjectWeeksTab, WeeklyReviewSubNav, WeekTimeline, MyWeekPage, ReviewsPage, and TeamMode. The fix replaced text-accent with text-sky-400 (#38bdf8), which achieves approximately 7:1 contrast while preserving the blue accent intent; the tradeoff is a slightly lighter blue than the original design token, but the gain in readability is substantial and the hue family remains consistent. The second pattern was Tailwind opacity modifiers (text-muted/50, text-muted/60) applied to already-muted gray text in DashboardVariantC.tsx timeline labels and prompt cards, which dropped contrast below the 4.5:1 floor; removing the modifiers and using the base text-muted value restored compliance without any visual change at normal reading distance. The third pattern was light red text (text-red-100, text-red-200) on a red-600 background in the Dashboard overdue-items banner, which only reached 3.95:1; switching to text-white resolved the contrast issue and improved scannability of the warning state. After these changes, the automated re-audit returned zero violations across all 13 pages, with Lighthouse accessibility scores averaging 100 and a minimum of 98.

Table 7. Accessibility compliance report.

Metric	Baseline	Current
Lighthouse accessibility score	Login: 98 My Week: 96 Dashboard: 96 Documents: 100 Issues: 100 Projects: 100 Programs: 100 Team Directory: 100 Team Allocation: 96 Status Overview: 96 Org Chart: 100 Reviews: 100 Settings: 100	Login: 98 My Week: 100 Dashboard: 100 Documents: 100 Issues: 100 Projects: 100 Programs: 100 Team Directory: 100 Team Allocation: 100 Status Overview: 100 Org Chart: 100 Reviews: 100 Settings: 100
Total critical/serious violations	26 total (1 critical, 25 serious)	0 total

Keyboard navigation completeness	Partial (allocation, directory, org-chart)	Partial
Color contrast failures	25	0
Missing ARIA labels or roles	Settings: tr:nth-child(1) > td:nth-child(3) > select	0

Reflections

- **Which parts of the audit were AI tools most helpful for? Least helpful?**
    - AI tools were very helpful for creating structured outputs for the benchmarks and comparing results between runs. With the caveat I had to be very specific about how the report should be structured and that the benchmarks measured the outcomes I was interested in (eg, API and SQL baselines). Also, it handled the type safety errors, bundle size efficiency, and accessibility features very easily, and with little guidance. But for the more complex optimizations I often found the AI tools would try to solve the more simple problem rather than the problem I identified and wanted to fix. For example when trying to optimize the slowest DB query, it would actually try to reduce the query count, rather than to fix the query pattern itself.

- **Did AI tools help you understand the codebase, or did they shortcut understanding?**
    - The AI tools helped me to understand the codebase. If I didn't have the AI tools it would be significantly more difficult to understand the dataflows in the application. Also, being able to talk through the design and suggested changes was immeasurably helpful.

- **Where did you have to override or correct AI suggestions? Why?**
    - Recurrently, I would have to override the AI when I was trying to fix the user facing issues (step 6). For example with the title not syncing between concurrent users, I saw that it was trying to change the API to use websockets, instead of the prebuilt CRUD path. The Issue was much more simple, a caching fix, and didn't require the more extensive code changes it wanted to implement. It seems to occur when the model can't actually test its changes through the browser, but thinks it fixed the issue at a conceptual level. To overcome this I would either run the end to end test myself or have it use the Playwright MCP to check its own work.
- **What percentage of your final code changes were AI-generated vs. hand-written?**
    - Code was entirely written by AI. I monitored the overall changes to make sure the changes were hitting the expected files. But if changes were needed I did it through the AI.

Discovery

Unified Document Model. The unified document model is a single PostgreSQL documents table that stores every content type in the application, wikis, issues, programs, projects, sprints, and people — differentiated only by a document_type ENUM column. I found this in api/src/db/schema.sql (lines 98-162) and its TypeScript counterpart in shared/src/types/document.ts (lines 34-44, 236-317). Type-specific data like an issue's priority or a project's ICE scores live in a schema-less properties JSONB column, while all documents share the same content (TipTap JSON) and yjs_state (CRDT binary) columns. Relationships between documents use a document_associations junction table (lines 208-222) with typed relationships like parent, project, sprint, and program, replacing what would traditionally be separate foreign key columns on separate tables. This matters because it eliminates an entire class of schema migrations, adding a new document type requires no table changes, just a new enum value and a TypeScript properties interface. In a future project, I would apply this pattern any time I'm building a system where multiple content types share behavior (editing, permissions, search) but differ in metadata, since it dramatically reduces the surface area of both the database schema and the API layer.

Discriminated Union. A discriminated union is when TypeScript uses a shared field — like document_type — to automatically narrow down which shape an object has. I found this in shared/src/types/document.ts (lines 34-44 and 268-317). The codebase defines a DocumentType as a union of string literals ('wiki' | 'issue' | 'program' | 'project' | ...), then creates specific interfaces like IssueDocument and ProjectDocument that each lock document_type to one specific string and pair it with a matching properties type. The powerful part is what TypeScript does with this: if you write if (doc.document_type === 'issue'), the compiler instantly knows that doc.properties has fields like state, priority, and assignee_id — no casting required. Without this pattern, you'd need unsafe type assertions or runtime checks scattered everywhere. This matters because it turns what would be runtime bugs ("I tried to access .priority on a wiki document") into compile-time errors that your editor catches as you type. In a future project, I would use discriminated unions any time I have objects that share a common structure but vary in details based on a "kind" or "type" field — API responses with different shapes, UI state machines, or event systems.

TipTap Editor Extensions. TipTap is a rich text editor library built on top of ProseMirror, and what makes it interesting is its plugin architecture: instead of getting one monolithic editor, you compose your editor from a list of independent extensions, each adding a specific capability. I found the full extension list in web/src/components/Editor.tsx (lines 552-618), where the editor is assembled from roughly 25 extensions — StarterKit provides basics like bold, italic, and paragraphs; CodeBlockLowlight adds syntax-highlighted code blocks; Table, TableRow, TableCell, and TableHeader add table support; TaskList and TaskItem add interactive checklists; and custom extensions like ImageUploadExtension, DocumentEmbed, and HypothesisBlockExtension add app-specific features. Each extension is configured independently with .configure({}), passing in only the options that extension cares about. The codebase also builds its own extensions using factory functions — for example,

createSlashCommands() in web/src/components/editor/SlashCommands.tsx (lines 266-315) takes app-level callbacks like onCreateSubDocument and returns a fully configured extension that powers the "/" menu, where each slash command is just a plain object with a title, description, aliases for search, and a command function that manipulates the editor. This plugin-based composition pattern matters because it means you never touch the editor's core code to add features — you just add another extension to the array. In a future project, I would look for this same compose-from-plugins architecture whenever choosing a library (editors, form builders, build tools), because it lets a codebase grow without the core getting more complicated.

LLM Usage

Plans: Claude Code Max ($200/month), and Codex ($20/month)

Model	**Input Tokens**	**Output Tokens**	**Cost**
Codex	31M	1M	$184.93
Claude	65,437	159,083	$49.08
Total			$234.01